

Üüritung — Estonian Rental Market Monitoring Platform

Team Members: Hashim | Sudais | Calvin | Daboikiabo

University of Tartu — Spring 2026

1. System Functionalities

The following functional requirements describe the key operations supported by the Üüritung platform. They are derived from the system proposal in Assignment 1 and serve as the basis for identifying the domain concepts and relationships in the domain model.

- R1.** The system shall scrape rental listings from Estonian portals (KV.ee, City24, Rendin) on a configurable schedule.
- R2.** The system shall normalize scraped listing data (price, size, rooms, neighborhood, address) into a unified schema and deduplicate across sources.
- R3.** When a new listing is detected, the system shall publish a listing.new event to the message broker.
- R4.** The system shall allow an administrator to manually trigger a scrape and view scrape job status.
- R5.** The system shall compute daily price analytics (average price, average price per square meter, listing count) per neighborhood.
- R6.** The system shall provide price trend data over a configurable time range for any tracked neighborhood.
- R7.** The system shall provide a city-wide summary including total listings, cheapest listing, most expensive listing, and average price.
- R8.** The system shall allow a user to register an account with name, email, and role (tenant or landlord).
- R9.** The system shall allow a user to update their profile information and saved search preferences.
- R10.** The system shall validate user identity and role when requested by other services.
- R11.** The system shall allow a tenant to create an alert rule specifying neighborhood, maximum price, and minimum size.
- R12.** When a new listing matches an active alert rule, the system shall trigger a notification to the alert owner via the Notification Service.
- R13.** The system shall allow a user to deactivate or delete an alert rule.
- R14.** The system shall maintain neighborhood profiles with descriptions, amenity scores, and transport information.
- R15.** The system shall allow users to submit reviews for neighborhoods they have lived in.
- R16.** The system shall provide neighborhood comparisons using analytics data and resident reviews.

- R17.** The system shall allow tenants to submit reviews for landlords, including a rating and text.
- R18.** The system shall compute a reputation score per landlord from aggregated review data.
- R19.** The system shall allow a landlord to respond to tenant reviews.
- R20.** The system shall allow a landlord to claim a scraped listing as their own property by providing verification details.
- R21.** The system shall allow a landlord to post a new direct listing with property details and photos.
- R22.** The system shall allow a landlord to update listing status (available, rented, withdrawn).
- R23.** The system shall deliver email and in-app notifications to users when triggered by other services.
- R24.** The system shall track notification delivery status (pending, sent, failed) and allow users to view their notification history.
- R25.** The system shall support configurable notification preferences per user (opt-in/opt-out, channel selection).

2. Domain Model Diagram

The domain model diagram below describes all domain concepts, their DDD tactical classifications (Aggregate Root, Entity, Value Object, Repository), attributes, and relationships across the eight microservices. Each concept indicates which service owns it. Cross-service references use identifiers only — no cross-database joins occur.

The following is a full textual specification of the domain model diagram. It is structured as a UML class diagram where each class box shows the stereotype (e.g., <<Aggregate Root>>), the class name with owning service, and its attributes. Repository interfaces are shown as separate boxes. Lines between classes represent relationships with multiplicity labels. Dashed lines indicate cross-service references by identifier.

Scraper Service [Hashim]

<<Aggregate Root>> Listing [ScraperService]

- listingId : UUID (PK)
- source : Enum (KV_EE, CITY24, RENDIN)
- externalId : String
- title : String
- price : Money (Value Object)
- size : BigDecimal (m²)
- pricePerSqm : BigDecimal (derived)
- rooms : Integer
- neighborhood : String
- address : Address (Value Object)
- url : String
- scrapedAt : DateTime
- isActive : Boolean

<<Value Object>> Money [ScraperService]

- amount : BigDecimal
- currency : String (EUR)

<<Value Object>> Address [ScraperService]

- street : String
- city : String
- postalCode : String

<<Entity>> ScrapeJob [ScraperService]

- jobId : UUID (PK)
- source : Enum (KV_EE, CITY24, RENDIN)
- startedAt : DateTime
- completedAt : DateTime
- status : Enum (RUNNING, COMPLETED, FAILED)
- listingsFound : Integer
- newListings : Integer

<<interface>> IListingRepository

(provides persistence access for the Listing aggregate)

Relationships:

- Listing contains Money (composition, 1..1) — price is embedded in Listing
- Listing contains Address (composition, 1..1) — address is embedded in Listing
- ScrapeJob produces Listing (association, 1..* — one job finds many listings)
- IListingRepository manages Listing

Analytics Service [Hashim]

<<Aggregate Root>> NeighborhoodSnapshot [AnalyticsService]

- snapshotId : UUID (PK)

- neighborhood : String
- date : Date
- averagePrice : BigDecimal
- averagePricePerSqm : BigDecimal
- medianPrice : BigDecimal
- listingCount : Integer
- priceChangePercent : BigDecimal

<<Value Object>> DateRange [AnalyticsService]

- startDate : Date
- endDate : Date

<<interface>> INeighborhoodSnapshotRepository

(provides persistence access for the NeighborhoodSnapshot aggregate)

Relationships:

- NeighborhoodSnapshot uses DateRange (composition, 1..1) — date range for trend queries
- NeighborhoodSnapshot references Listing (cross-service, by neighborhood String — no FK)
- INeighborhoodSnapshotRepository manages NeighborhoodSnapshot

User Service [Sudais]

<<Aggregate Root>> User [UserService]

- userId : UUID (PK)
- firstName : String
- lastName : String
- email : String
- phone : String
- role : Enum (TENANT, LANDLORD, ADMIN)
- contactInfo : ContactInfo (Value Object)
- createdAt : DateTime
- updatedAt : DateTime

<<Value Object>> ContactInfo [UserService]

- email : String
- phone : String

<<Entity>> SavedSearch [UserService]

- searchId : UUID (PK)
- userId : UUID (FK)
- neighborhood : String
- maxPrice : BigDecimal
- minSize : BigDecimal
- minRooms : Integer
- createdAt : DateTime

<<interface>> IUserRepository

(provides persistence access for the User aggregate)

Relationships:

- User contains ContactInfo (composition, 1..1) — embedded value object
- User has SavedSearch (composition, 1..* — a user can have many saved searches)
- User is referenced by AlertRule.userId (cross-service, Alert Service)
- User is referenced by LandlordProfile.userId (cross-service, Landlord Service)
- User is referenced by Notification.recipientUserId (cross-service, Notification Service)
- IUserRepository manages User

Alert Service [Sudais]

<<Aggregate Root>> AlertRule [AlertService]

- alertId : UUID (PK)
- userId : UUID (references User in UserService)
- neighborhood : String (optional — null means any)
- maxPrice : BigDecimal
- minSize : BigDecimal (optional)
- minRooms : Integer (optional)
- isActive : Boolean
- createdAt : DateTime

<<Entity>> AlertMatch [AlertService]

- matchId : UUID (PK)
- alertId : UUID (FK)
- listingId : UUID (references Listing in ScraperService)
- matchedAt : DateTime
- notified : Boolean

<<interface>> IAlertRuleRepository

(provides persistence access for the AlertRule aggregate)

Relationships:

- AlertRule has AlertMatch (composition, 1..* — one rule can produce many matches)
- AlertRule references User.userId (cross-service, User Service — for alert ownership)
- AlertMatch references Listing.listingId (cross-service, Scraper Service — the matched listing)
- IAlertRuleRepository manages AlertRule

Neighborhood Service [Calvin]

<<Aggregate Root>> Neighborhood [NeighborhoodService]

- neighborhoodId : UUID (PK)
- name : String
- city : String
- description : String
- location : GeoCoordinate (Value Object)
- amenityScore : BigDecimal
- transportScore : BigDecimal
- safetyRating : BigDecimal

<<Value Object>> GeoCoordinate [NeighborhoodService]

- latitude : Double
- longitude : Double

<<Entity>> NeighborhoodReview [NeighborhoodService]

- reviewId : UUID (PK)
- neighborhoodId : UUID (FK)
- userId : UUID (references User in UserService)
- rating : Integer (1-5)
- text : String
- createdAt : DateTime

<<interface>> INeighborhoodRepository

(provides persistence access for the Neighborhood aggregate)

Relationships:

- Neighborhood contains GeoCoordinate (composition, 1..1) — location embedded
- Neighborhood has NeighborhoodReview (composition, 1..* — many reviews per neighborhood)
- NeighborhoodReview references User.userId (cross-service, User Service — reviewer identity)
- Neighborhood references NeighborhoodSnapshot (cross-service, Analytics Service — for price data display)
- INeighborhoodRepository manages Neighborhood

Landlord Service [Calvin]

<<Aggregate Root>> LandlordProfile [LandlordService]

- landlordId : UUID (PK)
- userId : UUID (references User in UserService)
- displayName : String
- verificationStatus : Enum (PENDING, VERIFIED, REJECTED)
- averageRating : BigDecimal (derived)
- totalReviews : Integer (derived)
- createdAt : DateTime

<<Entity>> TenantReview [LandlordService]

- reviewId : UUID (PK)
- landlordId : UUID (FK)
- reviewerUserId : UUID (references User in UserService)
- rating : Integer (1-5)
- text : String
- landlordResponse : String (optional)
- createdAt : DateTime

<<interface>> ILandlordProfileRepository

(provides persistence access for the LandlordProfile aggregate)

Relationships:

- LandlordProfile has TenantReview (composition, 1..* — many reviews per landlord)
- LandlordProfile references User.userId (cross-service, User Service — landlord identity)
- TenantReview references User.userId (cross-service, User Service — reviewer identity)
- LandlordProfile is referenced by ManagedListing.landlordId (cross-service, Listing Service)
- ILandlordProfileRepository manages LandlordProfile

Listing Service [Daboikiabo]

<<Aggregate Root>> ManagedListing [ListingService]

- managedListingId : UUID (PK)
- landlordId : UUID (references LandlordProfile in LandlordService)
- scrapedListingId : UUID (optional, references Listing in ScraperService)
- title : String
- description : String
- price : Money (Value Object)
- size : BigDecimal
- rooms : Integer
- address : Address (Value Object)
- status : Enum (AVAILABLE, RENTED, WITHDRAWN)
- isVerified : Boolean
- createdAt : DateTime
- updatedAt : DateTime

<<Entity>> ClaimRequest [ListingService]

- claimId : UUID (PK)
- landlordId : UUID (FK)
- scrapedListingId : UUID (references Listing in ScraperService)
- status : Enum (PENDING, APPROVED, REJECTED)
- verificationDetails : String
- submittedAt : DateTime
- resolvedAt : DateTime

<<interface>> IManagedListingRepository

(provides persistence access for the ManagedListing aggregate)

Relationships:

- ManagedListing contains Money (composition, 1..1) — price embedded
- ManagedListing contains Address (composition, 1..1) — address embedded
- ManagedListing has ClaimRequest (composition, 0..1 — a listing may have one claim)
- ManagedListing references LandlordProfile.landlordId (cross-service, Landlord Service)
- ManagedListing references Listing.listingId (cross-service, Scraper Service — for claimed listings)
- ClaimRequest references Listing.listingId (cross-service, Scraper Service)
- IManagedListingRepository manages ManagedListing

Notification Service [Daboikiabo]

<<Aggregate Root>> Notification [NotificationService]

- notificationId : UUID (PK)
- recipientUserId : UUID (references User in UserService)
- channel : Enum (EMAIL, IN_APP)
- subject : String
- body : String
- status : Enum (PENDING, SENT, FAILED)
- sentAt : DateTime
- createdAt : DateTime

<<Entity>> NotificationTemplate [NotificationService]

- templateId : UUID (PK)
- name : String
- subjectTemplate : String
- bodyTemplate : String

<<Entity>> NotificationPreference [NotificationService]

- preferenceId : UUID (PK)
- userId : UUID (references User in UserService)
- channel : Enum (EMAIL, IN_APP)
- enabled : Boolean

<<interface>> INotificationRepository

(provides persistence access for the Notification aggregate)

Relationships:

- Notification uses NotificationTemplate (association, *..1 — many notifications can use one template)
- Notification references User.userId (cross-service, User Service — recipient identity)
- NotificationPreference references User.userId (cross-service, User Service)
- Notification is triggered by AlertMatch (cross-service, Alert Service — via async event)
- INotificationRepository manages Notification

3. Domain Model Summary

The table below summarizes each service's domain concepts and their DDD tactical classification.

Service (Owner)	Aggregate Root	Entities	Value Objects	Repository
Scraper Service (Hashim)	Listing	ScrapeJob	Money, Address	IListingRepository
Analytics Service (Hashim)	Neighborhood Snapshot	—	DateRange	INeighborhoodSnapshotRepository
User Service (Sudais)	User	SavedSearch	ContactInfo	IUserRepository
Alert Service (Sudais)	AlertRule	AlertMatch	—	IAlertRuleRepository
Neighborhood Svc (Calvin)	Neighborhood	Neighborhood Review	GeoCoordinate	INeighborhoodRepository
Landlord Service (Calvin)	LandlordProfile	TenantReview	—	ILandlordProfileRepository
Listing Service (Daboikiabo)	ManagedListing	ClaimRequest	Money, Address (reused VOs)	IManagedListingRepository
Notification Svc (Daboikiabo)	Notification	Notification Template, Notification Preference	—	INotificationRepository

4. Cross-Service Relationships

The following table documents all cross-service relationships in the domain model. Each relationship uses identifier references only — no shared databases or cross-service foreign keys exist.

From Concept [Service]	To Concept [Service]	Relationship	Reference Type
AlertRule [AlertService]	User [UserService]	Alert belongs to User	userId (UUID ref)
AlertMatch [AlertService]	Listing [ScraperService]	Match references matched Listing	listingId (UUID ref)
NeighborhoodReview [NeighborhoodSvc]	User [UserService]	Review written by User	userId (UUID ref)
Neighborhood [NeighborhoodSvc]	NeighborhoodSnapshot [AnalyticsService]	Profile displays price data	neighborhood (String ref)
LandlordProfile [LandlordService]	User [UserService]	Landlord linked to User account	userId (UUID ref)
TenantReview [LandlordService]	User [UserService]	Review written by User	reviewerUserId (UUID ref)
ManagedListing [ListingService]	LandlordProfile [LandlordService]	Listing owned by Landlord	landlordId (UUID ref)
ManagedListing [ListingService]	Listing [ScraperService]	Claimed scraped listing	scrapedListingId (UUID ref)
Notification [NotificationSvc]	User [UserService]	Notification sent to User	recipientUserId (UUID ref)
NotificationPref [NotificationSvc]	User [UserService]	Preference owned by User	userId (UUID ref)

5. Individual Contributions

Hashim — Scraper Service & Analytics Service

Responsible for the Listing aggregate (Aggregate Root + ScrapeJob Entity + Money/Address Value Objects + IListingRepository) and the NeighborhoodSnapshot aggregate (Aggregate Root + DateRange Value Object + INeighborhoodSnapshotRepository). Cross-service integration: NeighborhoodSnapshot references Listing data by neighborhood; Listing publishes events consumed by AlertRule (Sudais) and is referenced by ManagedListing (Daboikiabo).

Sudais — User Service & Alert Service

Responsible for the User aggregate (Aggregate Root + SavedSearch Entity + ContactInfo Value Object + IUserRepository) and the AlertRule aggregate (Aggregate Root + AlertMatch Entity + IAlertRuleRepository). Cross-service integration: User is referenced by five other services as the central identity provider; AlertRule consumes listing.new events from Scraper (Hashim) and triggers Notification (Daboikiabo).

Calvin — Neighborhood Service & Landlord Service

Responsible for the Neighborhood aggregate (Aggregate Root + NeighborhoodReview Entity + GeoCoordinate Value Object + INeighborhoodRepository) and the LandlordProfile aggregate (Aggregate Root + TenantReview Entity + ILandlordProfileRepository). Cross-service integration: Neighborhood references NeighborhoodSnapshot (Hashim) for price data; LandlordProfile is referenced by ManagedListing (Daboikiabo) and reviews reference User (Sudais).

Daboikiabo — Listing Service & Notification Service

Responsible for the ManagedListing aggregate (Aggregate Root + ClaimRequest Entity + Money/Address reused Value Objects + IManagedListingRepository) and the Notification aggregate (Aggregate Root + NotificationTemplate + NotificationPreference Entities + INotificationRepository). Cross-service integration: ManagedListing references LandlordProfile (Calvin) and Listing (Hashim); Notification references User (Sudais) and is triggered by AlertMatch (Sudais).

6. DDD Tactical Elements Checklist

DDD Element	Count	Examples
Aggregate Roots	8	Listing, NeighborhoodSnapshot, User, AlertRule, Neighborhood, LandlordProfile, ManagedListing, Notification
Entities	8	ScrapeJob, SavedSearch, AlertMatch, NeighborhoodReview, TenantReview, ClaimRequest, NotificationTemplate, NotificationPreference
Value Objects	5	Money, Address, DateRange, ContactInfo, GeoCoordinate
Repositories	8	IListingRepository, INeighborhoodSnapshotRepository, IUserRepository, IAlertRuleRepository, INeighborhoodRepository, ILandlordProfileRepository, IManagedListingRepository, INotificationRepository
Cross-service relationships	10	See Section 4 — all use identifier references (UUID or String), no shared databases